

Reviewer Pack — Minimal Invariant Length of Coxeter Groups and Resolution Pr...

Entry 533c01fc264a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 00:12:16 UTC

Minimal Invariant Length of Coxeter Groups and Resolution Proof Depth

Entry ID: 533c01fc264a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 00:12:16 UTC

1. Conjecture

Field A (mathematical branch): Coxeter Group Theory **Field B** (complexity object): Resolution Proof Complexity

Statement:

{'main': 'For any instance of a Tseitin formula on an expander graph, the resolution proof depth is upper bounded by $O(2^{(\delta(G) + c)})$, where $\delta(G)$ is the minimal number of generators of the Coxeter group that represents the symmetry group of G .', 'counterexample': 'Provide an instance of a Tseitin formula on an expander graph for which the resolution proof depth is less than $2^{(\delta(G) + c)}$.', 'constructive_mapping': '1. Construct the symmetry group of the given expander graph G using its automorphism group. 2. Represent the symmetry group as a Coxeter group using the minimal number of generators. 3. Calculate the resolution proof depth for the Tseitin formula corresponding to G .')}

Rationale (proposer's reasoning):

{'reason_1': 'Coxeter groups are algebraic structures that can represent symmetries in various contexts, and their properties might reflect the complexity of the graphs they describe.', 'reason_2': "The invariant length (number of generators) of a Coxeter group could potentially capture essential features of the graph's structure that affect its computational complexity, particularly in resolution proofs.", 'reason_3': 'This conjecture aims to bridge the gap between algebraic representation theory and the computational complexity of resolution proof systems.'}

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 42be98a7ad57d72d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Resolution proof depth of Tseitin formulas on expander graphs is within $O(2^{(\delta(G) + c)})$ where $\delta(G)$ is the minimal number of generators for the Coxeter group.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Coxeter Group Theory' AND 'resolution proof complexity' AND 'Tseitin formula' - 'minimal invariant length' AND 'Coxeter group' AND 'resolution depth' - 'expander graph' AND 'Coxeter group generators' AND 'proof complexity'

Top relevant hits considered: - [http://arxiv.org/abs/1405.3051v1] Involution Products in Coxeter Groups - [http://arxiv.org/abs/1405.2700v1] Zero Excess and Minimal Length in Finite Coxeter Groups - [http://arxiv.org/abs/0804.0947v2] The Dynkin diagram cohomology of finite Coxeter groups - [http://arxiv.org/abs/1712.08214v3] The length and depth of algebraic groups - [http://arxiv.org/abs/2010.03414v2] Topological boundaries of connected graphs and Coxeter groups - [http://arxiv.org/abs/2401.11068v2] Defining sequences for fundamental root systems and Coxeter graphs for super Weyl groups

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate an expander graph with n vertices and m edges
    n = 30
    m = 2 * (n - 1)
    G = {i: [] for i in range(n)}
    for _ in range(m):
        u, v = random.sample(range(n), 2)
        if v not in G[u]:
            G[u].append(v)
            G[v].append(u)

    # Calculate the minimal number of generators for the Coxeter group
    δ_G = len(G) # Simplified assumption: each vertex represents a generator

    # Construct Tseitin formula (simplified version)
    clauses = []
    for i in range(n):
```

```

        clauses.append([i + n])
    for j in G[i]:
        if j > i:
            clauses.append([-i - n, -j - n, i + 1 + n * j])

# Determine the resolution proof depth (simplified version)
def resolve(clauses):
    resolved = set()
    while True:
        new_clauses = []
        for clause in clauses:
            if len(clause) == 1 and clause[0] not in resolved:
                resolved.add(-clause[0])
            else:
                for other_clause in clauses:
                    if len(other_clause) == 1 and -other_clause[0] in clause:
                        new_clauses.extend([c for c in clause if c != -other_clause[0]])
                        break
        if not new_clauses:
            return len(resolved)
        clauses = new_clauses

proof_depth = resolve(clauses)

metric_value = proof_depth
conjecture_holds = proof_depth <= 2 ** (δ_G + 1) # Simplified constant c=1 for demonstration
counterexample = "" if conjecture_holds else "simplified_formula"

return {
    "metric_name": "Resolution Proof Depth",
    "metric_value": metric_value,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2**i + 3 for i in range(
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results) and support_fraction >= 0.8:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"simplified_formula\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence n_tested={len(results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e129446f.py", line 78, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e129446f.py", line 59, in run_trial
    proof_depth = resolve(clauses)
                  ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e129446f.py", line 48, in resolve
    if len(clause) == 1 and clause[0] not in resolved:
       ^^^^^^^^^
```

TypeError: object of type 'int' has no len()

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the resolution proof depth could not be calculated and therefore the conjecture cannot be supported or falsified based on this test. | next: Re-run the test with a different instance of a Tseitin formula on an expander graph to verify if the crash is reproducible and to obtain data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15796
2	preregistration	ollama_remote	glm4:latest	0	0	9797
3	novelty	ollama_remote	glm4:latest	0	0	8587
4	novelty	ollama_remote	glm4:latest	0	0	9266
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	37861
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6505
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7974
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9163
9	judge	ollama_remote	glm4:latest	0	0	11811

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 116759 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/533c01fc264a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/533c01fc264a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/533c01fc264a.tar.gz` (if generated)